

Student Record Management System

Week-01 | Task 1 | Khansa Malik

1. Introduction

A Student Record Management System was developed in Python to demonstrate practical software engineering practices in a complete, maintainable application. It provides student record management through a command-line interface (CLI) while separating presentation, business logic, data modelling, and persistence.

2. Objective

The objective was to implement a reliable student record application supporting CRUD operations, input validation, search, statistics, persistent storage, exception handling, logging, and automated testing.

3. System Design and Implementation

The application uses a layered architecture in which each module has a defined responsibility, reducing coupling and improving maintainability and testability.

Module	Responsibility
models.py	Defines the Student dataclass and validates name, age, email, and GPA.
storage.py	Provides JSONStorage and uses atomic writes to reduce the risk of file corruption.
manager.py	Contains business logic for CRUD, search, statistics, and specific application exceptions.
cli.py	Handles user input/output and delegates operations to the manager layer.

4. Key Features

- Full CRUD functionality for student records.
- Keyword search across student ID, name, and grade; total student count and average GPA.
- Automatic persistence to students.json after mutating operations.
- Defensive validation and specific exceptions for duplicate or missing records.
- Logging of significant operations and errors for traceability.

5. Testing and Verification

The system was tested with pytest through 19 automated unit tests covering model validation, CRUD success and failure cases, search, statistics, and persistence across separate StudentManager instances. All 19 tests passed successfully, including persistence verification after a simulated application restart.

6. Software Engineering Practices

The implementation applies separation of concerns, defensive validation, exception handling, logging, automated testing, and safe file persistence. The storage layer is isolated so JSON persistence can later be replaced with a database with minimal changes to the business logic.

7. Outcome and Conclusion

The completed system is a functional, tested CLI application with modular architecture and persistent storage. The task provided practical experience in structuring Python applications beyond a monolithic script and applying principles that improve reliability, maintainability, and future scalability.